

WebCig

Un archivio navigabile dei contratti pubblici italiani
Relazione di progetto

Massimiliano Ricchiuti
Elaborazione di testi e gestione documentale

A.A. 2025/2026

1. Obiettivo e corpus

WebCig trasforma fonti eterogenee relative ai contratti pubblici in documenti XML validati, estrae informazioni e genera un archivio HTML statico. Il corpus comprende 21 XML, uno per CIG, derivati dai CSV e JSON [ANAC](#) e dai documenti HTML e PDF conservati in `fonti_originali/`. Il catalogo `fonti_originali/web/fonti_web.json` aggiunge 33 risorse web, distribuite su 21 CIG. I conteggi sono aggiornati dalla pipeline mediante `report/generat`
`ed/consistenza.tex`.

La preparazione individua i CIG nei JSON e integra le righe corrispondenti del CSV, quando disponibili. Le altre fonti locali sono associate tramite il CIG nel nome del file oppure nel testo estraibile. Il catalogo web conserva URL, ente, formato, fase, sintesi, data di verifica e nesso con il contratto; il caricatore controlla metadati, URL e corrispondenza con i CIG del dataset. Non effettua una nuova verifica online durante ogni build.

2. Modello documentale e validazione

La radice `contratto` identifica il CIG e raccoglie fonti, dati di gara, stazione appaltante, incaricati, pubblicazioni, partecipanti, importi, date e descrizione. Le sezioni facoltative, fra cui esecuzione, quadro economico, categorie e CPV, dipendono dalle informazioni disponibili. La DTD `schema/contratto_cig.dtd` definisce ordine, cardinalità e attributi. Due modelli misti alternano testo e marcatori semantici ripetibili:

```
<!ELEMENT descrizioneDocumentale
  (#PCDATA | tipologia | oggetto | categoria | termine | luogo)*>
<!ELEMENT fonteWeb
  (#PCDATA | titoloFonte | enteFonte | faseFonte | nessoFonte | datoFonte)*>
```

`lxml.etree` esegue parsing e validazione; un XML mal formato o non conforme interrompe la build. Gli esiti sono registrati in `output_data/validation.xml`. Gli XPath estraggono il modello intermedio `ContractRecord`, mentre le viste documentali leggono direttamente gli XML e preservano elementi ripetuti, attributi, valori vuoti e ordine del contenuto misto.

3. Elaborazioni e consultazione

Le analisi calcolano copertura delle fonti, distribuzioni territoriali, sequenze cronologiche e statistiche economiche. Gli importi usano `Decimal`; le durate negative sono segnalate ed escluse dalle statistiche. `pypdf`, con ripiego su `pdfminer.six`, estrae il testo dei PDF. La pipeline genera JSON, CSV, grafici SVG e frammenti LaTeX.

Il *Lessico dei contratti* seleziona con XPath il solo `oggettoGara` di ogni XML. [spaCy](#) e il [modello italiano it_core_news_sm](#) raggruppano nomi comuni e aggettivi in lemmi, escludendo stopwords, entità riconosciute e token non alfabetici. I 20 lemmi più frequenti espongono forme, occorrenze e CIG distinti nel *Report dei dati*. Le concordanze conservano il testo originale e collegano il documento; si aprono con `details/summary`, anche senza JavaScript. Modello e criteri sono registrati nel JSON scaricabile.

Jinja2 genera il sito statico in `dist/`. La Home ha sei sezioni numerate e un indice persistente; *Archivio*, *Report dei dati* e *Qualità dei dati* completano la navigazione. L'Archivio offre ricerca, filtri e ordinamento. Ogni CIG presenta Scheda completa, Struttura XML e Documenti; XML e DTD sono scaricabili e le fonti web dichiarano il nesso. I dettagli includono microdata `schema.org/GovernmentService`. Tutti gli asset sono locali. Senza JavaScript rimangono accessibili contenuti, collegamenti e concordanze; ricerca, ordinamento e linguette richiedono JavaScript.

4. Limiti

Il campione è didattico e non rappresentativo. Campi assenti o non omogenei limitano il confronto; le incongruenze non sono corrette arbitrariamente e le differenze fra importi non sono automaticamente ribassi. Il riconoscimento dei documenti richiede il CIG nel nome o nel testo: manca OCR per i PDF immagine. Le fonti web possono mutare e un nesso di contesto non equivale a un riscontro sul CIG esatto. La lemmatizzazione può sbagliare su sigle e linguaggio amministrativo; le frequenze non costituiscono una classificazione tematica.

5. Riproducibilità e verifiche

Dalla radice del repository, con Python 3.11 o successivo, su Linux/macOS:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
python scripts/build_all.py
python -m pytest
```

Su Windows PowerShell usare `py -3 -m venv .venv` e `.venv\Scripts\Activate.ps1`. `requirements-dev.txt` include le dipendenze del progetto. Per ricompilare il PDF serve una distribuzione LaTeX con `pdflatex`; il pacchetto Python omonimo non installa il compilatore. In sua assenza la build del sito usa il PDF già presente. `scripts/build.py` elabora gli XML esistenti; `scripts/build_report.py` ricompila la sola relazione. Dopo quest'ultimo comando occorre rigenerare il sito per aggiornare il download. L'anteprima si avvia con `python -m http.server 8000 -directory dist`.

I test controllano parsing, DTD, modelli misti, fonti, estrazione, statistiche, viste XML, microdata, pagine e collegamenti interni. Per il lessico verificano conteggi e corrispondenza degli estratti agli XML. Il controllo manuale di servizio/servizi, impianto/impianti e fornitura/forniture ha verificato il raggruppamento; l'errore su arredi è corretto esplicitamente in arredo. Le dipendenze includono spaCy e il modello con versioni fissate. Il workflow prepara il dataset, costruisce il sito, esegue i test e pubblica `dist/` su `main`, anche per l'avvio manuale. Le modifiche di `development` richiedono il merge in `main`. Il PDF versionato viene copiato senza compilare LaTeX. Le verifiche non certificano l'esattezza amministrativa delle fonti.

6. Prompt documentati

Questa sezione incorpora gli otto prompt e le note già registrati nel progetto e aggiunge la sintesi delle revisioni successive. Le formulazioni storiche sono conservate: `DOCS_DIR` appartiene a una richiesta iniziale; l'implementazione attuale configura la directory tramite `ProjectPaths.xml_dir`. Gli strumenti generativi sono stati usati per supportare modellazione, codice e documentazione; gli output richiedono controlli sui sorgenti e sui risultati. Le note dei prompt 6–8 documentano gli interventi precedenti, senza attribuire loro verifiche nuove.

Prompt 1. Analisi della struttura del progetto

Analizza le specifiche del progetto TEAM e verifica che la struttura del progetto distingua correttamente tra fonti originali, directory XML di input, DTD, script Python, output HTML/CSS e report finale. Evidenzia eventuali elementi non conformi e proponi correzioni minime senza introdurre requisiti non presenti nelle specifiche.

Prompt 2. Modellazione XML dei CIG

A partire da un CSV [ANAC](#) con CIG e da JSON di dettaglio disponibili per alcuni CIG, progetta un modello XML descrittivo per rappresentare un singolo contratto pubblico. Il modello deve evitare una semplice struttura tabellare, deve includere contenuto misto, deve mantenere il collegamento alle fonti originali e deve poter essere validato con DTD.

Prompt 3. DTD e validazione

Dato il modello XML dei contratti pubblici, costruisci una DTD semplice ma coerente. Alcune sezioni devono essere opzionali perché non tutti i CIG hanno JSON dettagliato, aggiudicazione, CPV o PDF. La descrizione documentale deve consentire contenuto misto con elementi interni come tipologia, oggetto, categoria, termine e luogo.

Prompt 4. Allineamento agli esempi di laboratorio

Riscrivi gli script Python in modo che seguano l'impostazione degli esempi di laboratorio: uso di `lxml.etree`, variabile `DOCS_DIR`, scansione della directory XML, parsing, validazione DTD e generazione di HTML. Integra inoltre uno script di verifica microdata e uno script di analisi PDF con librerie viste nel corso.

Prompt 5. Revisione del report

Revisiona il report LaTeX del progetto affinché descriva in modo sintetico la fase di preparazione, la directory XML di input, la validazione, gli output prodotti, le librerie Python utilizzate e i limiti del lavoro. Il testo deve restare nel perimetro del corso e non trasformarsi in una relazione sugli appalti pubblici.

6. Prompt documentati (segue)

Prompt 6. Rifattorizzazione, analisi e pubblicazione statica

È stato richiesto di analizzare integralmente il progetto, separare il codice di supporto dalle elaborazioni, introdurre un modello intermedio dei record, implementare analisi territoriali, cronologiche ed economiche, trasformare l'output in un sito statico responsivo e accessibile, predisporre la pubblicazione automatizzata, ampliare il report LaTeX e aggiungere test automatici.

Gli output generati sono stati verificati mediante esecuzione della pipeline completa, validazione DTD, confronto dei conteggi delle fonti, controllo automatico dei collegamenti nella directory `dist/`, test `pytest`, verifica dei microdata e compilazione LaTeX quando disponibile. Le metriche del report non sono state trascritte manualmente: derivano da `output_data/analysis.json` e dai frammenti in `report/generated/`.

Prompt 7. Ricerca e qualificazione delle fonti web

Per ogni CIG presente nell'archivio, reperisci documenti, determine o altre fonti web autorevoli riferibili alla gara. Le risorse devono arricchire le fonti originarie e consolidare il content model misto. Evita soluzioni limitate a un CIG: conserva provenienza, formato, fase e criterio di collegamento, distinguendo un riscontro esatto da una fonte relativa a lotto, accordo quadro, CUP, fase antecedente o contesto.

Le risorse selezionate sono state registrate in un manifesto dati e integrate nella pipeline comune. La verifica non si limita alla raggiungibilità del collegamento: il nesso dichiarato evita di attribuire a una gara un atto che documenta soltanto il suo contesto. Per i procedimenti relativi a minori è stata inoltre privilegiata la minimizzazione, collegando fonti istituzionali senza ripubblicare documenti potenzialmente sensibili.

Prompt 8. Aggiornamento del corpus su development

Dopo l'aggiunta dei documenti locali, cercare autonomamente i riferimenti istituzionali per i nuovi CIG, aggiornare le fonti web e ricompilare il progetto affinché i documenti compaiano nell'archivio. Operare esclusivamente sul branch `development`.

I sei nuovi CIG sono stati confrontati con i JSON e le determine locali. Sono state aggiunte sette risorse istituzionali, distinguendo l'accordo quadro di Verona dal suo primo contratto applicativo. Le date di verifica precedenti sono conservate; il problema di accesso remoto al PDF di Cremona è dichiarato nel riscontro. La build ha rigenerato XML, sito, analisi e report, con validazione DTD e controlli automatici.

Prompt 9. Revisione della Home e integrazione della relazione

Sintesi della richiesta del 17 settembre 2026. Operare sul branch `development`; rendere navigabili le sezioni della Home, rimuovere il riepilogo della collezione e l'anteprima laterale, adottare il titolo WebCig richiesto e collegare [ANAC](#) alla pagina della Piattaforma contratti pubblici. Rimuovere dalla Home le sezioni sull'associazione ai CIG e sui criteri di provenienza; rinominare Archivio, Report dei dati e istruzioni di compilazione. Rendere la relazione coerente con il progetto, integrarvi i prompt e rimuovere il documento separato e il relativo collegamento.

L'intervento aggiorna i template e la navigazione, mantiene le analisi e i documenti del corpus e concentra descrizione e prompt in questa relazione, scaricabile dalla sezione Documentazione. La numerazione e i collegamenti della Home funzionano come ancore della pagina statica.

Prompt 10. Lessico e concordanze

Sintesi della richiesta del 18 settembre 2026. Aggiungere su `main` l'analisi linguistica con lemmi e concordanze nel Report dei dati, mantenendo struttura e codice semplici. L'intervento seleziona gli oggetti di gara, applica spaCy durante la compilazione e mostra frequenze e riscontri originali in sezioni espandibili. Sono documentati criteri, limiti e controlli; il sito rimane statico.